



U N I V E R S I D A D
COMPLUTENSE
M A D R I D

Funciones recursivas

Enrique Martín (emartinm@ucm.es)
Programación Declarativa (PD)
Grado en Ingeniería Informática – 3º
Fac. Informática

Contenidos

- 1 La idea de recursión
- 2 Recursión sobre números
- 3 Recursión sobre listas
- 4 Recursión múltiple y mutua
- 5 Acumuladores
- 6 Quicksort
- 7 Pensar de forma recursiva

- G. Hutton. *Programming in Haskell* (2ª ed.). Cambridge University Press, 2016.
 - Capítulo 6: *Recursive Functions*

La idea de recursión

Definir una función en términos de sí misma

- Una función es **recursiva** cuando su definición se refiere a sí misma
- Ya hemos usado la recursión, sin nombrarla explícitamente, desde la Sesión 2:

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

- Hoy la estudiamos como técnica **general** para definir funciones, y la comparamos con los bucles imperativos

Caso base y caso recursivo

- Toda función recursiva bien definida tiene, como mínimo, dos tipos de ecuación:
 - El **caso base**: un caso sin llamada recursiva, que detiene el proceso
 - El **caso recursivo**: un caso que se define en términos de una versión “más pequeña” del mismo problema
- En `factorial`: el caso base es `factorial 0 = 1`; el caso recursivo reduce el problema de `n` al problema (más pequeño) de `n - 1`
- Sin caso base, o sin que el problema decrezca, la recursión **no termina**

Recursión vs. bucles imperativos

- En un lenguaje imperativo, un bucle recorre un rango de valores modificando variables:

```
def factorial(n):  
    res = 1  
    while n > 1:  
        res = res * n  
        n = n - 1  
    return res
```

- En Haskell, sin variables mutables ni bucles, la **recursión es el mecanismo natural** para repetir un cálculo un número variable de veces
- Cada llamada recursiva juega el papel de “una vuelta más” del bucle, y el caso base juega el papel de la condición de salida

Recursión sobre números

- Definamos x^n para $n \geq 0$ por recursión sobre el exponente:

```
potencia :: Int -> Int -> Int
potencia _ 0 = 1
potencia x n = x * potencia x (n - 1)
```

```
ghci> potencia 2 10
1024
```

- Caso base: cualquier número elevado a 0 es 1. Caso recursivo:
 $x^n = x \cdot x^{n-1}$, exactamente como la definición matemática

Máximo común divisor: el algoritmo de Euclides

- El algoritmo de Euclides calcula el MCD usando que $\text{mcd}(a, b) = \text{mcd}(b, a \bmod b)$, hasta llegar a resto 0:

```
mcd :: Int -> Int -> Int
mcd a 0 = a
mcd a b = mcd b (a `mod` b)
```

```
ghci> mcd 48 18
6
```

- Aquí el problema no decrece simplemente restando 1, sino que decrece porque el segundo argumento se hace cada vez más pequeño hasta llegar a 0

¿Por qué termina mcd?

- No basta con tener un caso base: hay que asegurarse de que el caso recursivo **siempre se acerca** a él
- En $\text{mcd } a \ b = \text{mcd } b \ (a \ \text{mod} \ b)$, el segundo argumento de la llamada recursiva es $a \ \text{mod} \ b$, que es **siempre estrictamente menor** que b (cuando $b > 0$)
- Como los naturales no pueden decrecer indefinidamente, el segundo argumento llega a 0 en un número finito de pasos, activando el caso base

Suma de los primeros n naturales

```
sumaHasta :: Int -> Int
sumaHasta 0 = 0
sumaHasta n = n + sumaHasta (n - 1)
```

```
ghci> sumaHasta 100
5050
```

- Comparar con la fórmula cerrada $\frac{n(n+1)}{2}$: la versión recursiva es menos eficiente, pero ilustra perfectamente el patrón caso base + caso recursivo sobre números

Recursión sobre listas

Reimplementando length

```
miLength :: [a] -> Int
miLength []      = 0
miLength (_:xs) = 1 + miLength xs
```

```
ghci> miLength [1,2,3,4]
4
```

- Caso base: la lista vacía tiene longitud 0
- Caso recursivo: la longitud es 1 (por la cabeza) más la longitud del resto

Reimplementando sum y product

```
miSum :: [Int] -> Int
miSum []      = 0
miSum (x:xs) = x + miSum xs
```

```
miProduct :: [Int] -> Int
miProduct []      = 1
miProduct (x:xs) = x * miProduct xs
```

- Observa el paralelismo: ambas funciones comparten exactamente la misma **estructura** recursiva, y solo cambian el caso base (elemento neutro) y el operador combinador (+ o *)
- Esta observación es la semilla de **foldr**, que veremos en la sesión sobre funciones de orden superior

Reimplementando (++)

```
miApend :: [a] -> [a] -> [a]
miApend []      ys = ys
miApend (x:xs) ys = x : miApend xs ys
```

```
ghci> miApend [1,2,3] [4,5]
[1,2,3,4,5]
```

- La recursión se hace sobre el **primer** argumento; el segundo, *ys*, no cambia en ninguna llamada
- El caso base dice que concatenar la lista vacía con *ys* da simplemente *ys*

Reimplementando reverse

```
miReverse :: [a] -> [a]
miReverse []      = []
miReverse (x:xs) = miReverse xs ++ [x]
```

```
ghci> miReverse [1,2,3]
[3,2,1]
```

- Es la misma definición que ya vimos en la Sesión 4 al presentar **reverse** del prelude
- Volveremos sobre ella más adelante en esta sesión: es **ineficiente**, y veremos cómo mejorarla con un acumulador

Reimplementando take y drop

```
miTake :: Int -> [a] -> [a]
miTake 0 _      = []
miTake _ []     = []
miTake n (x:xs) = x : miTake (n-1) xs
```

```
miDrop :: Int -> [a] -> [a]
miDrop 0 xs      = xs
miDrop _ []     = []
miDrop n (_:xs) = miDrop (n-1) xs
```

- Aquí la recursión decrece en **dos** sentidos a la vez: el contador n y la propia lista, y cualquiera de los dos puede agotarse primero

Recursión sobre dos listas a la vez

- Hasta ahora hemos recursionado siempre sobre **una** lista; también se puede recursionar sobre **dos** listas a la vez, encajando patrones sobre ambas:

```
miZip :: [a] -> [b] -> [(a,b)]
miZip [] _          = []
miZip _ []          = []
miZip (x:xs) (y:ys) = (x,y) : miZip xs ys
```

```
ghci> miZip [1,2,3] "ab"
[(1,'a'),(2,'b')]
```

- Aquí hay **tres** casos: cualquiera de las dos listas vacía (caso base, dos ecuaciones) y ambas no vacías (caso recursivo)
- El problema decrece porque **ambas** listas, *xs* e *ys*, son más cortas que las originales; este patrón reaparecerá al recorrer árboles, más adelante en la asignatura

Recursión múltiple y mutua

Más de una llamada recursiva: Fibonacci

- La sucesión de Fibonacci se define con **dos** llamadas recursivas en el caso general:

```
fib :: Int -> Int
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

```
ghci> fib 10
55
```

- Aquí hay **dos** casos base (0 y 1), porque el caso recursivo necesita los dos valores anteriores

El coste oculto de fib

- Esta definición de `fib`, aunque directa, es muy **ineficiente**: recalcula los mismos valores muchísimas veces
- Por ejemplo, `fib 5` llama a `fib 4` y `fib 3`; pero `fib 4` vuelve a llamar a `fib 3` (¡otra vez!) y a `fib 2`
- El número de llamadas crece **exponencialmente** con n , aunque solo hay $n + 1$ resultados distintos que calcular

Recursión mutua: par e impar

- Dos funciones son **mutuamente recursivas** cuando cada una llama a la otra:

```
esPar :: Int -> Bool
esPar 0 = True
esPar n = esImpar (n - 1)
```

```
esImpar :: Int -> Bool
esImpar 0 = False
esImpar n = esPar (n - 1)
```

```
ghci> esPar 10
True
```

- Cada llamada alterna entre las dos funciones, pero el argumento sigue decreciendo en cada paso, así que el proceso termina igual que con una sola función recursiva

Acumuladores

¿Por qué es lenta nuestra versión de reverse?

- Recordemos la definición de `miReverse`:

```
miReverse (x:xs) = miReverse xs ++ [x]
```

- Cada llamada hace una concatenación `++`, cuyo coste es proporcional a la longitud de la lista de la **izquierda**
- Al final, se realizan n concatenaciones, cada una más cara que la anterior: el coste total es **cuadrático** ($O(n^2)$), no lineal

La idea del acumulador

- Un **acumulador** es un parámetro adicional que va guardando el resultado **parcial** calculado hasta el momento, en lugar de combinarlo al volver de la recursión
- En vez de esperar a que vuelvan todas las llamadas recursivas para combinar resultados (como hace ++ en `miReverse`), el acumulador se actualiza **antes** de cada llamada recursiva
- Esto permite, en muchos casos, pasar de coste cuadrático a coste lineal

reverse con acumulador

```
reverseAcc :: [a] -> [a] -> [a]
reverseAcc acc []      = acc
reverseAcc acc (x:xs) = reverseAcc (x:acc) xs
```

```
reverseRapido :: [a] -> [a]
reverseRapido xs = reverseAcc [] xs
```

```
ghci> reverseRapido [1,2,3]
[3,2,1]
```

- En cada paso, se añade la cabeza x al **principio** del acumulador (coste constante, $(:)$ no recorre nada), en lugar de al final con $++$
- El coste total es ahora lineal: $O(n)$

Recursión de cola (*tail recursion*)

- Una llamada recursiva es de **cola** cuando es la **última** operación que se realiza, sin nada pendiente por hacer al volver de ella
- `reverseAcc` es de cola: tras la llamada recursiva no queda ninguna operación pendiente

```
reverseAcc acc (x:xs) = reverseAcc (x:acc) xs
```

- `miReverse` **no** lo es: tras volver de la llamada recursiva todavía falta hacer `++ [x]`

```
miReverse (x:xs) = miReverse xs ++ [x]
```

- Las llamadas de cola no necesitan “recordar” nada pendiente, así que algunos lenguajes (y, bajo ciertas condiciones, GHC) pueden ejecutarlas con espacio de pila constante. Volveremos a este término al estudiar Prolog

Quicksort

La idea de quicksort

- **Quicksort** ordena una lista eligiendo un elemento **pivote** y dividiendo el resto en dos grupos: los menores y los mayores (o iguales) que el pivote
- Cada grupo se ordena **recursivamente** de la misma forma, y el resultado final es: (menores ordenados) ++ pivote ++ (mayores ordenados)
- El caso base es la lista vacía (y, trivialmente, la lista de un elemento)

Quicksort en Haskell

```
quicksort :: Ord a => [a] -> [a]
quicksort []      = []
quicksort (p:xs) =
    quicksort menores ++ [p] ++ quicksort mayores
  where
    menores = filter (< p) xs
    mayores = filter (>= p) xs
```

```
ghci> quicksort [3,1,4,1,5,9,2,6]
[1,1,2,3,4,5,6,9]
```

- Una definición de apenas cinco líneas, gracias a combinar recursión, `filter` y `++`

Traza de quicksort [3,1,4,1,5]

```
quicksort [3,1,4,1,5]
= quicksort [1,1] ++ [3] ++ quicksort [4,5]

quicksort [1,1] = quicksort [] ++ [1] ++ quicksort [1]
                = [] ++ [1] ++ [1]
                = [1,1]

quicksort [4,5] = quicksort [] ++ [4] ++ quicksort [5]
                = [] ++ [4] ++ [5]
                = [4,5]

quicksort [3,1,4,1,5] = [1,1] ++ [3] ++ [4,5]
                    = [1,1,3,4,5]
```

- El pivote de cada llamada (3, luego 1 y 4) divide la lista en dos partes cada vez más pequeñas, hasta llegar a la lista vacía

Ordenación por inserción: una alternativa

- Otra estrategia recursiva clásica es la **ordenación por inserción**: insertar cada elemento en su posición dentro de la lista ya ordenada del resto:

```
insertar :: Ord a => a -> [a] -> [a]
insertar x []      = [x]
insertar x (y:ys)
  | x <= y      = x : y : ys
  | otherwise   = y : insertar x ys
```

```
insercion :: Ord a => [a] -> [a]
insercion []      = []
insercion (x:xs) = insertar x (insercion xs)
```

Pensar de forma recursiva

Pautas para diseñar una función recursiva

1. Decide el **tipo** de la función (te dice sobre qué estructura vas a recursionar: un número, una lista, ...)
2. Enumera los **casos posibles** de la entrada (p. ej. 0 y n; o [] y (x:xs))
3. Define el (los) **caso(s) base** de forma directa, sin recursión
4. Define el **caso recursivo**, asumiendo que la llamada recursiva sobre el subproblema más pequeño **ya funciona correctamente**
5. Comprueba que el subproblema de cada llamada recursiva es realmente **más pequeño** (para garantizar que termina)

Error habitual: olvidar el caso base

- Si se omite el caso base, la función nunca termina (o falla con un patrón no cubierto):

```
factorialMalo :: Int -> Int
factorialMalo n = n * factorialMalo (n - 1)
```

```
ghci> factorialMalo 5
-- se queda calculando indefinidamente
```

- GHC **no detecta** este error: comprobar que una función termina es, en general, un problema indecidible (relacionado con el problema de la parada)

Error habitual: el problema no decrece

- Otro error típico es que la llamada recursiva no reduzca realmente el problema:

```
sumaHastaMala :: Int -> Int
sumaHastaMala n = n + sumaHastaMala n  -- (!) falta n-1
```

- Aquí el argumento de la llamada recursiva es **el mismo** n , así que nunca se alcanza el caso base
- Conviene, al escribir cualquier función recursiva, preguntarse explícitamente: *¿por qué este programa termina?*, igual que hicimos con `mcd` y con `quicksort`